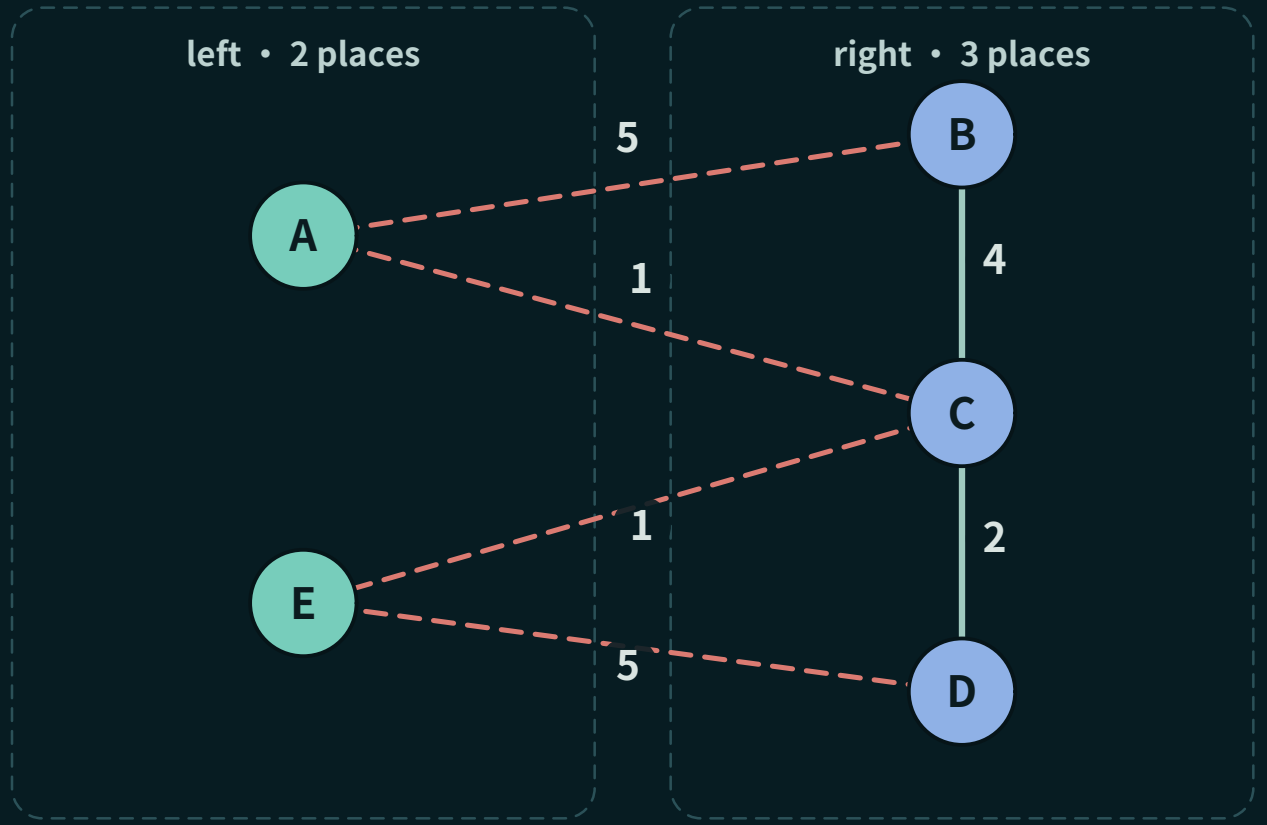
Try, then roll back

Kernighan–Lin refinement · pair swaps and the best prefix

Learning Clustering · original author FanqinZeng (Issue #6)

The problem

- Input: an undirected weighted graph and an initial bisection with fixed side sizes
- Move: swap one pair, a (left) and b (right)
- Goal: make the cut (total weight across) smaller
- Constraint: the node count per side never changes
- Output: the trial sequence, the best prefix, the accepted grouping and the stop reason



A fixed head count ≠ area balance: with cells of different areas, swapping two nodes can unbalance the area.

Seed AE | BCD: cut = 12

Background and place

Take a decent grouping, then improve it step by step.

KL was proposed by Kernighan and Lin in 1970 (Bell System Technical Journal), a classic heuristic for improving a bisection.

In this course it follows Module 06: spectral bisection gives the initial grouping and KL improves it locally; Module 08's FM then replaces the pair swap with single moves.

The cut and D values

- D(v) = external weight − internal weight
- External: v's edges to the other side; internal: v's edges to its own side
- D(v) is how much the cut drops if v alone moves across
- The larger D, the more v "wants" to cross

Node	Side	Ext.	Int.	D = ext − int
A	L	6	0	+6
B	R	5	4	+1
C	R	2	6	-4
D	R	5	2	+3
E	L	6	0	+6

The pair gain

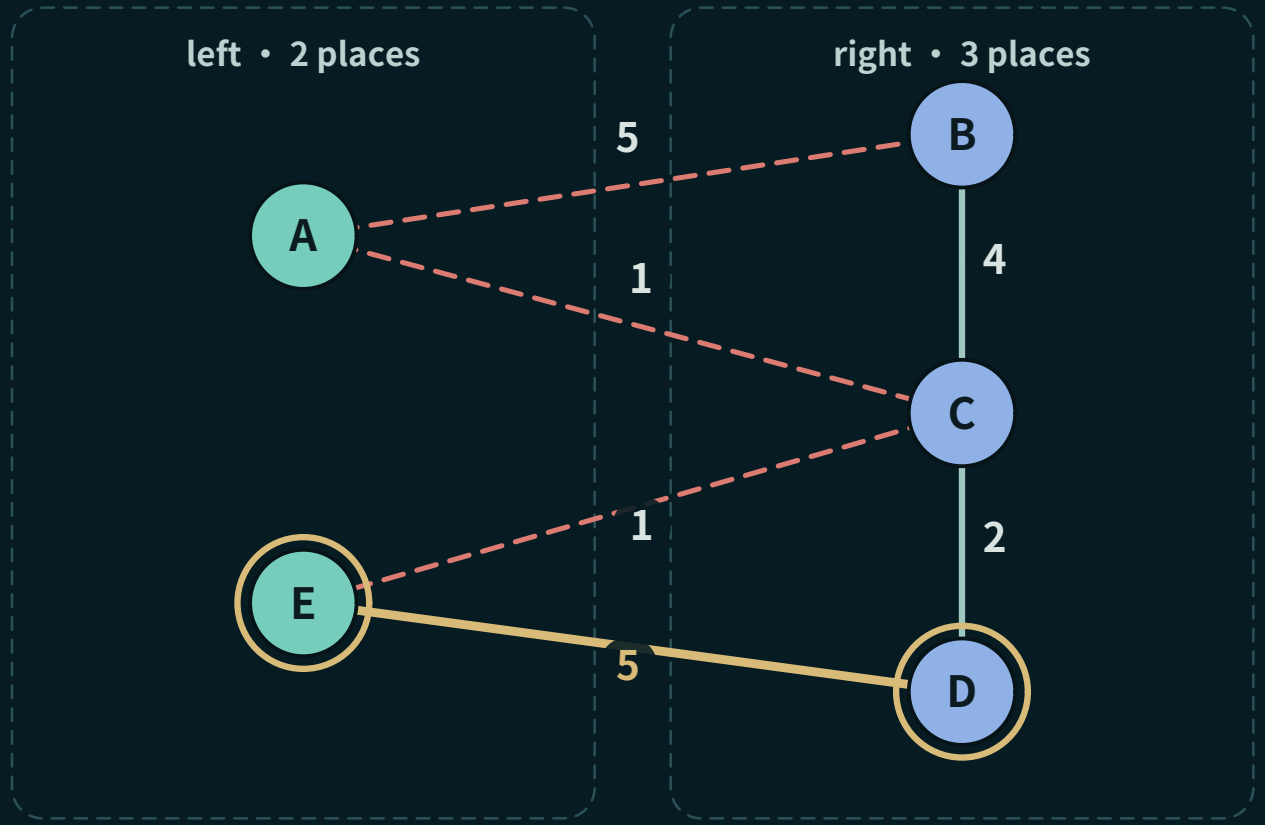
- The gain of swapping a, b: $g = D(a) + D(b) - 2w(a,b)$
- One node from each side: $2 \times 3 = 6$ pairs
- Largest: A↔D, $6 + 3 - 0 = 9$
- Ties go by the lexicographic order of $(-g, a, b)$

$$g(a,b) = D(a) + D(b) - 2w(a,b)$$

left a	right b	D(a)+D(b)	−2w	g
A	D	9	0	+9
E	B	7	0	+7
A	C	2	-2	0
E	C	2	-2	0
E	D	9	-10	-1
A	B	7	-10	-3

Why subtract 2w

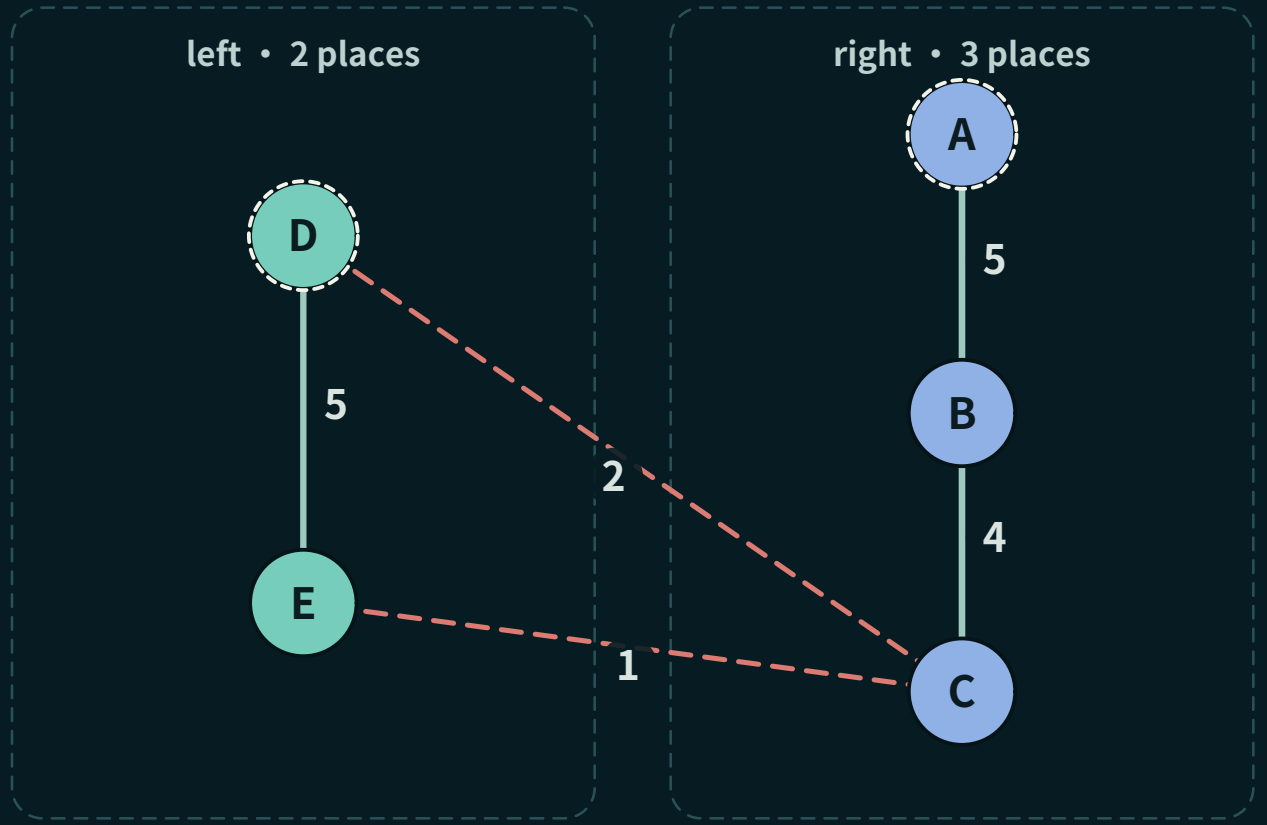
- D(E) + D(D) = 6 + 3 = 9
- Both D values count edge DE (weight 5) as "external"
- After the swap E and D are still on opposite sides: DE is still cut
- The extra must come back off: 2 × 5 = 10
- g(E, D) = 9 − 10 = −1



E↔D: DE is cut before and after → take back 2 × 5

Step one: a trial swap, then lock

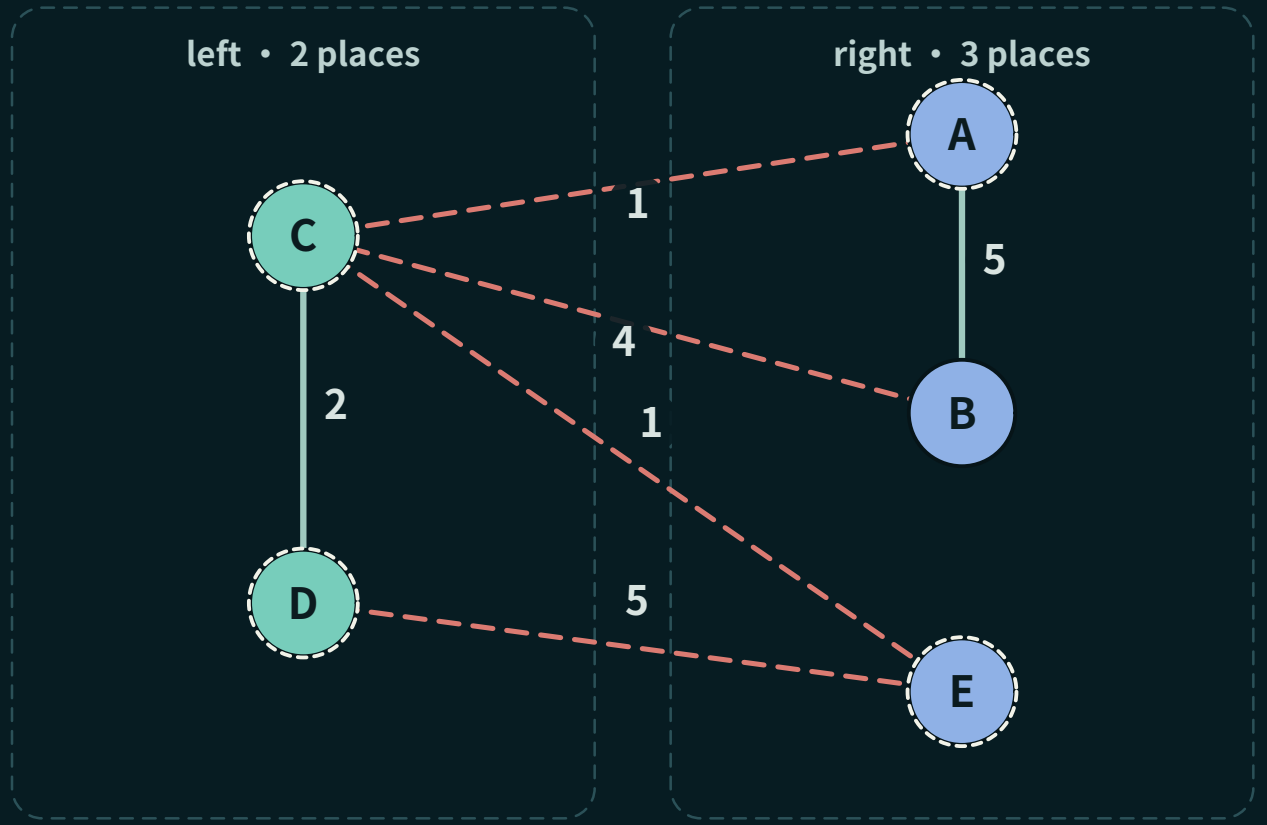
- Pick the largest-gain pair: A↔D, g = +9
- After the swap, cut 12 → 3
- A and D lock: they don't move again this pass (dashed rings)
- This is still a "trial", not yet accepted



Trial A↔D: g = +9, cut 12 → 3

Step two: take the negative gain too

- Recompute D: the candidates left are E↔C −8, E↔B −13
- KL does not stop at the first negative gain
- It still takes the largest, E↔C: the cut rises to 11 for now
- No unlocked node is left on the left: the pass's trials end

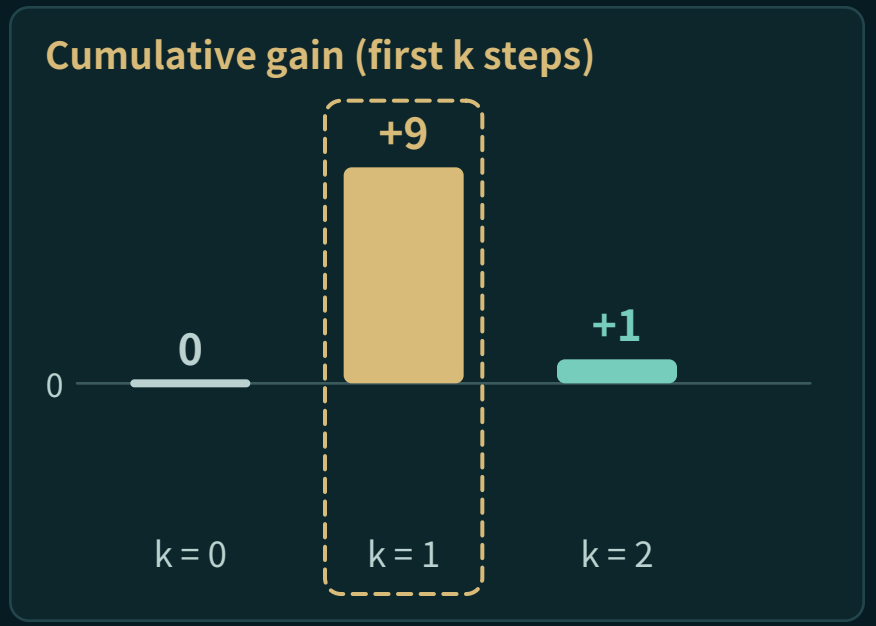


Why take a negative gain? On a bigger graph, a step that first makes things worse can be followed by a larger positive gain; only a finished pass shows it.

Trial E↔C: g = −8, the cut rises to 11 for now

The best prefix and roll-back

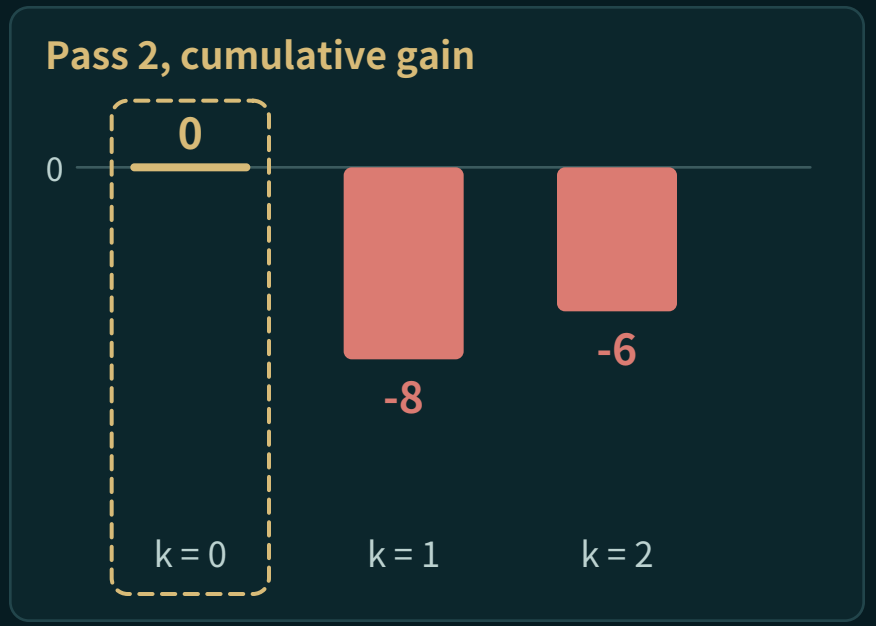
- Cumulative gains: k=0 → 0, k=1 → +9, k=2 → +1
- Take the largest, consecutive prefix only: k* = 1
- From the pass start, apply only the first step
- Accept ABC | DE, cut = 3; step 2 is undone



Accept the first 1: ABC | DE, cut 3

Pass two: the empty prefix and the stop

- Start from ABC | DE, everything unlocked
- Trials: E↔C −8, D↔B +2
- Cumulative −8, −6: neither beats doing nothing
- The best prefix is empty (k = 0) → the algorithm stops



The empty prefix k = 0 wins → stop, cut 3

The pseudocode, line by line

- Lines 1 and 8: pass after pass, until the best prefix is empty
- Lines 3–5: take negative gains too; after a swap, lock and update D
- Lines 6–7: roll back to the start and apply only the best prefix
- Each node swaps at most once per pass

```
1  repeat:                  # one pass each time
2    start from the current split, all unlocked
3    while both sides have unlocked nodes:
4      pick the unlocked (a, b) with the largest g
5      swap a, b (a trial), lock, update D
6    take the prefix k with the largest total (k = 0 allowed)
7    from the pass start apply only the first k
8  until k = 0
```

Common mistakes

- Forgetting −2w: computing g(E,D) as +9 and thinking E↔D pays
- Stopping at a negative gain: ending pass 1 after cut 3, without seeing every prefix
- Taking "any positive steps" as the prefix: a prefix is consecutive from step 1
- Reporting the trial state: cut 11 instead of the rolled-back 3
- Thinking count balance is area balance

Scope and limits

- KL is a heuristic: cut 3 on this graph does not mean it finds the global optimum in general
- The result depends on the initial grouping: compare methods from the same seed
- Each step scans every cross pair: about $O(n^2)$ per step, naively
- It keeps only the node count per side; different cell areas need their own constraint
- In EDA it is often the local-improvement step of partitioning

Summary

A trial may make things worse; acceptance looks only at the best prefix.

Pair gain $g = D(a) + D(b) - 2w(a,b)$; lock each pass, roll back, repeat; stop on the empty prefix.

Next module, FM: move one node at a time, with a balance window instead of "one for one".